



Data Structure

Padmaraj Nidagundi



Last class.....

- 
- **What is data ?**
 - **What is data structure ?**
 - **Why we need to learn data structure ?**

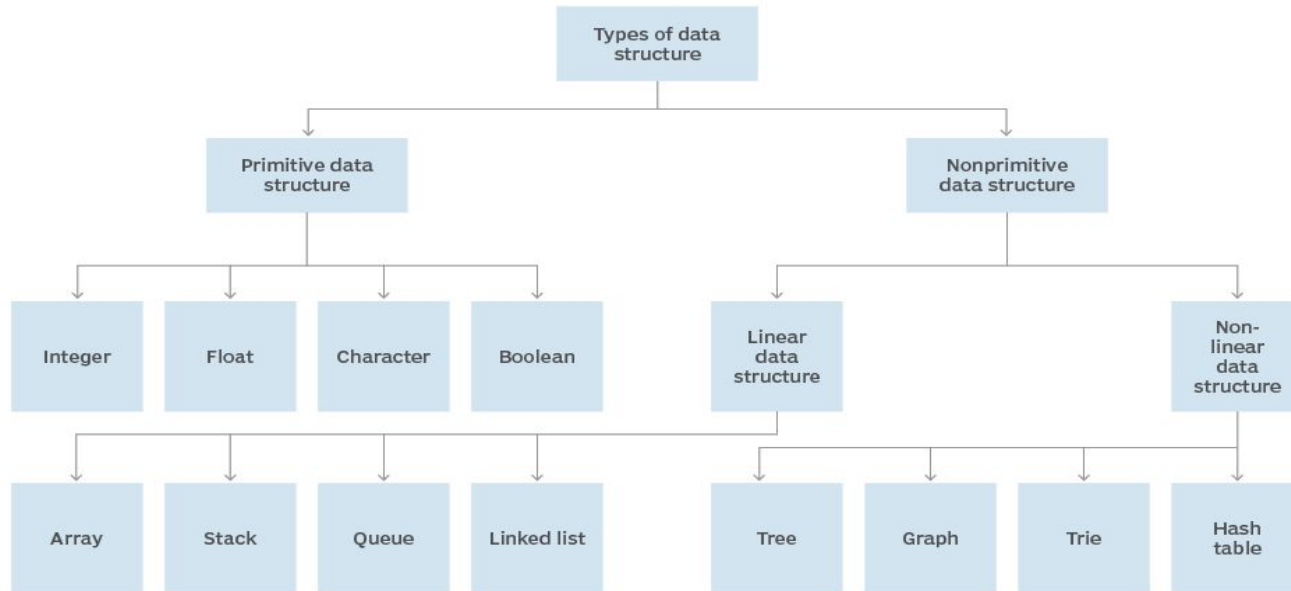


This class.....

- **Types of DS ?**

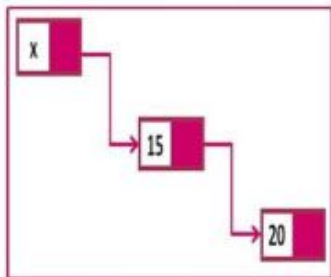
- **Importance of data structures in Programming?**
- **Basic Operations on DATA structure?**
- **Visualisation of Data structures**
- **Difference between Linear and Non-linear Data Structures**
- **Array in programming**

Data structure hierarchy

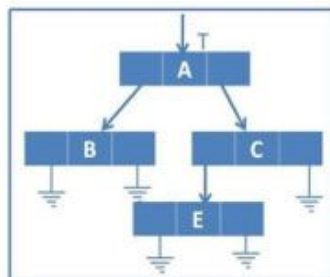




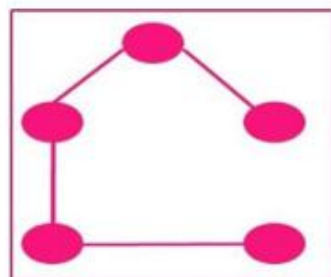
Sorting



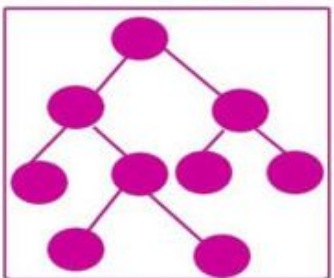
Link list



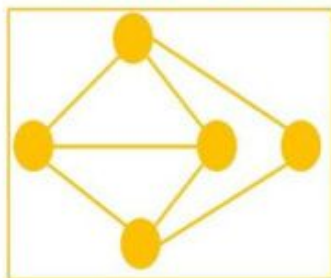
list



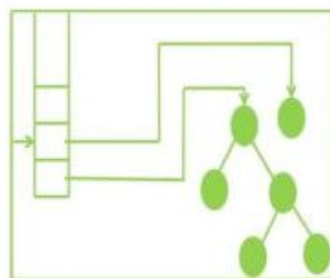
spanning tree



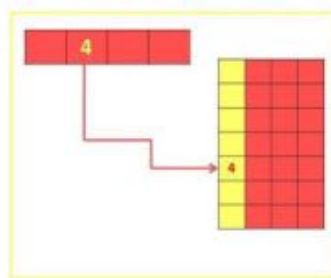
Tree



Graph



Stack



Hashing

Characteristic	Description
Linear	In Linear data structures, the data items are arranged in a linear sequence. Example: Array
Non-Linear	In Non-Linear data structures, the data items are not in sequence. Example: Tree, Graph
Homogeneous	In homogeneous data structures, all the elements are of same type. Example: Array
Non-Homogeneous	In Non-Homogeneous data structure, the elements may or may not be of the same type. Example: Structures
Static	Static data structures are those whose sizes and structures associated memory locations are fixed, at compile time. Example: Array
Dynamic	Dynamic structures are those which expands or shrinks depending upon the program need and its execution. Also, their associated memory locations changes. Example: Linked List created using pointers

Difference between Linear and Non-linear Data Structures

Key	Linear Data Structures	Non-linear Data Structures
Data Element Arrangement	In linear data structure, data elements are sequentially connected and each element is traversable through a single run.	In non-linear data structure, data elements are hierarchically connected and are present at various levels.
Levels	In linear data structure, all data elements are present at a single level.	In non-linear data structure, data elements are present at multiple levels.
Implementation complexity	Linear data structures are easier to implement.	Non-linear data structures are difficult to understand and implement as compared to linear data structures.
Traversal	Linear data structures can be traversed completely in a single run.	Non-linear data structures are not easy to traverse and needs multiple runs to be traversed completely.
Memory utilization	Linear data structures are not very memory friendly and are not utilizing memory efficiently.	Non-linear data structures uses memory very efficiently.
Time Complexity	Time complexity of linear data structure often increases with increase in size.	Time complexity of non-linear data structure often remain with increase in size.
Examples	Array, List, Queue, Stack.	Graph, Map, Tree.

Importance of data structures in Programming

- Data structures are essential for managing **large amounts of data**, such as information kept in **databases or indexing services**, efficiently.
- Proper maintenance of data systems requires the identification of **memory allocation, data interrelationships and data processes**
- Choosing an **ill-suited data structure could result in slow runtimes** or unresponsive code.
- A few factors to consider when picking a data structure include what **kind of information will be stored**, where should existing data be placed, how should data be sorted and how much memory should be reserved for the data.



Basic Operations on DATA structure

The data in the data structures are processed by certain operations. The particular data structure chosen largely depends on the frequency of the operation that needs to be performed on the data structure.

- **Traversing**
- **Searching**
- **Insertion**
- **Deletion**
- **Sorting**
- **Merging**

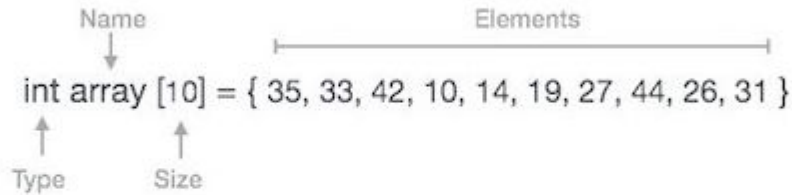


Break.....

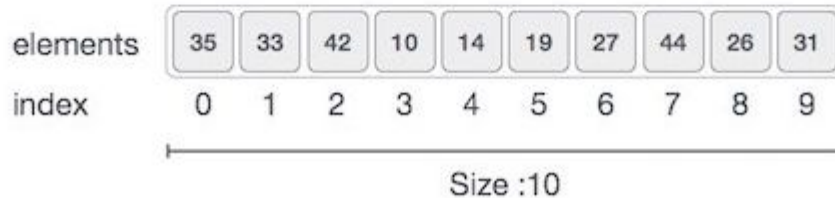
Array - The items of an array are allocated at adjacent memory locations. These memory locations are called elements of that array.

Array Representation

Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



Operations on Array



Basic Operations

Following are the basic operations supported by an array.

- ▣ **Traverse** – print all the array elements one by one.
- ▣ **Insertion** – Adds an element at the given index.
- ▣ **Deletion** – Deletes an element at the given index.
- ▣ **Search** – Searches an element using the given index or by the value.
- ▣ **Update** – Updates an element at the given index.



Why we need array, we should use variables ?



	0	1	2	3	4	5
Score	0.0	0.0	0.0	0.0	0.0	0.0

Array Storing Floating Values

When a program works with many variables which hold comparable forms of data, then organizational and managerial difficulty quickly arise.

If you are not using arrays, then the number of variables used will increase. Using the array, the number of variables reduces, i.e., you can use a single name for multiple values, you need to deal with its index values (starting from 0 to n).

So if the total run of each player is getting stored in separate variables, using arrays you can bring them all into one array having single name like:
`plrscore[11];`



Advantages

1. **Reading an array element is simple and efficient.** As shown in the above table, the read time of array is $O(1)$ in both best and worst cases. This is because any element can be instantly read using indexes (base address calculation behind the scene) without traversing the whole array.
2. **Array is a foundation of other data structures.** For example other data structures such as LinkedList, Stack, Queue etc. are implemented using array.
3. **All the elements of an array can be accessed using a single name** (array name) along with the index, which is readable, user-friendly and efficient rather than storing those elements in different-2 variables.



Disadvantages

1. While using array, we **must need to make the decision of the size of the array in the beginning**, so if we are not aware how many elements we are going to store in array, it would make the task difficult.
2. **The size of the array is fixed so if at later point, if we need to store more elements in it then it can't be done.** On the other hand, if we store less number of elements than the declared size, the remaining allocated memory is wasted.



How to declare an array?

Here's how you can declare an array in Java:

```
dataType[] arrayName;
```

- `dataType` can be a **primitive data** type like: int, char, Double, byte etc. or an object (will be discussed in later chapters).
- `arrayName` is an **identifier**.

Let's take the above example again.

```
double[] data;
```

Java Array Index

You can access elements of an array by using indices. Let's consider previous example.

```
int[] age = new int[5];
```



Array age of length 5

Array in Java - access elements of an array by using indices.

```
class ArrayExample {  
    public static void main(String[] args) {  
  
        int[] age = new int[5];  
  
        System.out.println(age[0]);  
        System.out.println(age[1]);  
        System.out.println(age[2]);  
        System.out.println(age[3]);  
        System.out.println(age[4]);  
    }  
}
```

0
0
0
0
0



Print elements of this array

```
class ArrayExample {  
    public static void main(String[] args) {  
  
        int[] age = {12, 4, 5, 2, 5};  
  
        for (int i = 0; i < 5; ++i) {  
            System.out.println("Element at index " + i + ": " + age[i]);  
        }  
    }  
}
```

```
Element at index 0: 12  
Element at index 1: 4  
Element at index 2: 5  
Element at index 3: 2  
Element at index 4: 5
```

Computes sum and average of values stored in an array of type int

```
class SumAverage {  
    public static void main(String[] args) {  
  
        int[] numbers = {2, -9, 0, 5, 12, -25, 22, 9, 8, 12};  
        int sum = 0;  
        Double average;  
  
        for (int number: numbers) {  
            sum += number;  
        }  
  
        int arrayLength = numbers.length;  
  
        // Change sum and arrayLength to double as average is in double  
        average = ((double)sum / (double)arrayLength);  
  
        System.out.println("Sum = " + sum);  
        System.out.println("Average = " + average);  
    }  
}
```

Sum = 36

Average = 3.6



Multidimensional Arrays

```
class MultidimensionalArray {  
    public static void main(String[] args) {  
  
        int[][] a = {  
            {1, 2, 3},  
            {4, 5, 6, 9},  
            {7},  
        };  
  
        System.out.println("Length of row 1: " + a[0].length);  
        System.out.println("Length of row 2: " + a[1].length);  
        System.out.println("Length of row 3: " + a[2].length);  
    }  
}
```

```
Length of row 1: 3  
Length of row 2: 4  
Length of row 3: 1
```




Visualisation of Data structures

<https://visualgo.net/en>

VISUALGO.NET/EN
visualising data structures and algorithms through animation



Home task

Write 5 different programs using array. Goal of task is to know more about array usage in the data structure programming.



End of class 2



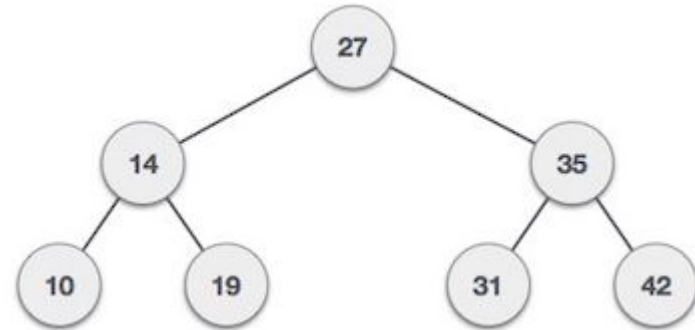


Binary search

Binary search is an efficient algorithm for finding an item from a **sorted list of items**. It works by repeatedly dividing in half the portion of the list that could contain the item, until you've narrowed down the possible locations to just one.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties

- The left sub-tree of a node has a key less than or equal to its parent node's key.
- The right sub-tree of a node has a key greater than to its parent node's key.





Basic Operations

- Search – Searches an element in a tree.
- Insert – Inserts an element in a tree.
- Pre-order Traversal – Traverses a tree in a pre-order manner.
- In-order Traversal – Traverses a tree in an in-order manner.
- Post-order Traversal – Traverses a tree in a post-order manner.



Binary Search Tree Visualization

<https://www.cs.usfca.edu/~galles/visualization/BST.html>



Advantages of using binary search tree

Searching become very efficient in a binary search tree since, we get a hint at each step, about which sub-tree contains the desired element.

The binary search tree is considered as **efficient data structure in compare to arrays and linked lists**. In searching process, it removes half sub-tree at every step. Searching for an element in a binary search tree takes $O(\log_2 n)$ time. In worst case, the time it takes to search an element is $O(n)$.

It also **speed up the insertion and deletion operations** as compare to that in array and linked list.

Binary Search - Insert operation



- **Step 1:** IF TREE = NULL
 - Allocate memory for TREE
 - SET TREE -> DATA = ITEM
 - SET TREE -> LEFT = TREE -> RIGHT = NULL
 - ELSE
 - IF ITEM < TREE -> DATA
 - Insert(TREE -> LEFT, ITEM)
 - ELSE
 - Insert(TREE -> RIGHT, ITEM)
 - [END OF IF]
 - [END OF IF]
- **Step 2:** END

Allocate the memory for tree.

- Set the data part to the value and set the left and right pointer of tree, point to NULL.
- If the item to be inserted, will be the first element of the tree, then the left and right of this node will point to NULL.
- Else, check if the item is less than the root element of the tree, if this is true, then recursively perform this operation with the left of the root.
- If this is false, then perform this operation recursively with the right sub-tree of the root.

Binary Search - Search operation

```
// A utility function to search a given key in BST
public Node search(Node root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root==null || root.key==key)
        return root;

    // val is greater than root's key
    if (root.key > key)
        return search(root.left, key);

    // val is less than root's key
    return search(root.right, key);
}
```

Searching a key

To search a given key in Binary Search Tree, we first compare it with root, if the key is present at root, we return root.

If key is greater than root's key, we recur for right subtree of root node.

Otherwise we recur for left subtree.